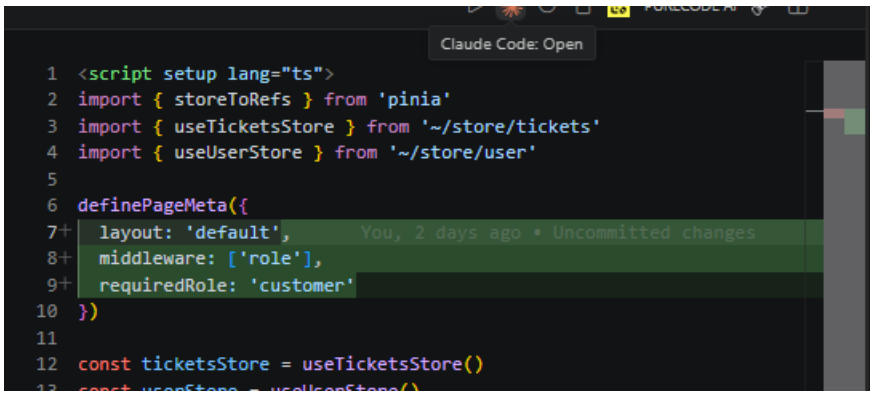
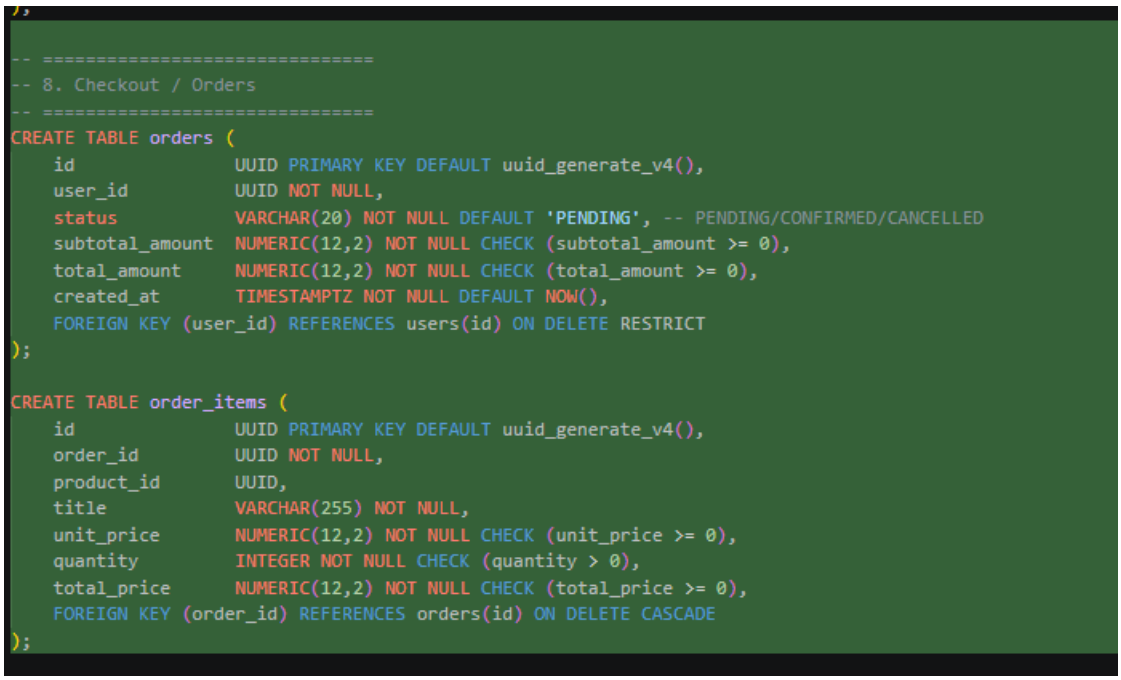


Hienj tại chỉ đnag mockdata để test



```
);

-- ==============================
-- 8. Checkout / Orders
-- ==============================
CREATE TABLE orders (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id         UUID NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'PENDING', -- PENDING/CONFIRMED/CANCELLED
    subtotal_amount NUMERIC(12,2) NOT NULL CHECK (subtotal_amount >= 0),
    total_amount    NUMERIC(12,2) NOT NULL CHECK (total_amount >= 0),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
);

CREATE TABLE order_items (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id        UUID NOT NULL,
    product_id      UUID,
    title           VARCHAR(255) NOT NULL,
    unit_price      NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
    quantity        INTEGER NOT NULL CHECK (quantity > 0),
    total_price     NUMERIC(12,2) NOT NULL CHECK (total_price >= 0),
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE
);
```

```python
class OrderItemCreate(BaseModel):
    product_id: str
    title: str
    quantity: int
    unit_price: float


class OrderItem(BaseModel):
    product_id: str
    title: str
    quantity: int
    unit_price: float
    total_price: float


class OrderCreateRequest(BaseModel):
    items: list[OrderItemCreate]


class OrderResponse(BaseModel):
    id: str
    user_email: str
    items: list[OrderItem]
    subtotal_amount: float
    total_amount: float
    status: str
    created_at: datetime


ORDERS: dict[str, OrderResponse] = {}


@contextmanager
def transaction():
    backup = ORDERS.copy()
    try:
        yield
    except Exception:
        ORDERS.clear()
        ORDERS.update(backup)
        raise
```

```python
@app.post("/api/orders/checkout", response_model=OrderResponse)
def checkout_order(
    payload: OrderCreateRequest,
    user: Annotated[UserRecord, Depends(get_current_user)],
):
    if len(payload.items) == 0:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="No order items provided")

    items = []
    for item in payload.items:
        if item.quantity <= 0 or item.unit_price < 0:
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Invalid order item")
        items.append(
            OrderItem(
                product_id=item.product_id,
                title=item.title,
                quantity=item.quantity,
                unit_price=item.unit_price,
                total_price=item.quantity * item.unit_price,
            )
        )

    subtotal = sum(item.total_price for item in items)
    if subtotal <= 0:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Invalid order total")

    with transaction():
        order_id = str(uuid4())
        order = OrderResponse(
            id=order_id,
            user_email=user.email,
            items=items,
            subtotal_amount=subtotal,
            total_amount=subtotal,
            status="PENDING",
            created_at=datetime.now(timezone.utc),
        )
        ORDERS[order_id] = order

    return order


@app.get("/api/orders/my", response_model=list[OrderResponse])
def get_my_orders(user: Annotated[UserRecord, Depends(get_current_user)]):
    return [order for order in ORDERS.values() if order.user_email == user.email]


@app.get("/api/orders/{order_id}", response_model=OrderResponse)
def get_order_by_id(order_id: str, user: Annotated[UserRecord, Depends(get_current_user)]):
    order = ORDERS.get(order_id)
    if not order:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Order not found")
    if order.user_email != user.email:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Forbidden")
    return order
```

```python
    response = client.get("/api/admin/dashboard")
    assert response.status_code == 401


def test_checkout_creates_order_and_returns_history():
    login_response = client.post(
        "/api/auth/login",
        json={"email": "customer@gmail.com", "password": "customer123"},
    )
    assert login_response.status_code == 200
    token = login_response.json()["access_token"]
    headers = {"Authorization": f"Bearer {token}"}

    checkout_response = client.post(
        "/api/orders/checkout",
        json={
            "items": [
                {
                    "product_id": "p1",
                    "title": "Test Movie Ticket",
                    "quantity": 2,
                    "unit_price": 50000,
                }
            ]
        },
        headers=headers,
    )
    assert checkout_response.status_code == 200
    order_data = checkout_response.json()
    assert order_data["user_email"] == "customer@gmail.com"
    assert order_data["subtotal_amount"] == 100000
    assert order_data["total_amount"] == 100000
    assert order_data["status"] == "PENDING"
    assert len(order_data["items"]) == 1

    history_response = client.get("/api/orders/my", headers=headers)
    assert history_response.status_code == 200
    history_data = history_response.json()
    assert isinstance(history_data, list)
    assert any(order["id"] == order_data["id"] for order in history_data)

    get_response = client.get(f"/api/orders/{order_data['id']}", headers=headers)
    assert get_response.status_code == 200
    assert get_response.json()["id"] == order_data["id"]
```